Київський національний університет

імені Тараса Шевченка

Звіт до лабораторної роботи 1:

”Пошук лідера у кільці”,

дисципліни: актуальні проблеми “Розподілені

системи обробки інформації”,

студента: Євчика Олексія,

групи Інформатика, Магістри

Тема

Алгоритм пошуку лідера у кільці розроблений Hirshberg та Sinclair (HS-алгоритм)

Мета роботи

Дослідити задачу вибору лідера в кільцевій топології розподіленої системи та реалізувати алгоритм Hirshberg–Sinclair для визначення процесу-лідера. Оцінити кількість раундів виконання алгоритму та сумарну кількість переданих повідомлень.

Завдання роботи

1. Ознайомитися з задачею вибору лідера у кільцевій мережі.
2. Вивчити принцип роботи алгоритму Hirshberg–Sinclair.
3. Реалізувати модель кільця процесів з унікальними UID.
4. Реалізувати передачу повідомлень у двох напрямках по кільцю.
5. Реалізувати фазову (раундову) логіку алгоритму.
6. Визначити процес-лідера.
7. Підрахувати:
    a. UID переможця
    b. кількість раундів
    c. сумарну кількість переданих повідомлень
8. Протестувати алгоритм для різної кількості процесів.

Теоретичні відомості

Задача вибору лідера полягає у визначенні одного процесу-координатора серед множини процесів у розподіленій системі. Кожен процес має унікальний ідентифікатор (UID), але не знає UID інших процесів. Вибір

лідера необхідний для координації роботи, прийняття глобальних рішень та управління ресурсами.

У кільцевій топології процеси організовані у логічне кільце. Кожен процес має лише двох сусідів:

- лівого
- правого

Передача повідомлень відбувається тільки між сусідніми процесами.

Алгоритм Hirshberg–Sinclair призначений для **синхронного кільця**, де обчислення виконуються раундами (фазами), а всі процеси діють паралельно. Кожен процес може знаходитися у двох станах:

- **active** - бере участь у виборах
- **passive** - вибуває після отримання більшого UID

Алгоритм працює фазами, у яких процеси перевіряють, чи є їх UID найбільшим. Основна ідея полягає у поступовому збільшенні області перевірки. У фазі k активний процес відправляє повідомлення в обох напрямках на відстань:

2^k

Експоненційне збільшення дистанції дозволяє:

- швидко охопити все кільце
- зменшити кількість фаз
- уникнути повного обходу кільця кожним процесом

Повідомлення містить:

- UID процесу

- напрямок передачі
- залишкову дистанцію

Поведінка процесу при отриманні повідомлення:

- якщо отриманий UID більший - процес стає passive і пересилає повідомлення
- якщо отриманий UID менший - повідомлення відкидається
- якщо процес отримує власний UID після повернення - він стає лідером

У кожній фазі кількість активних процесів зменшується. Після логарифмічної кількості фаз залишається лише один процес - з максимальним UID.

Алгоритм гарантує:

- вибір одного лідера
- завершення роботи
- коректність при унікальних UID

Складність алгоритму:

- кількість фаз: O(log n)
- кількість повідомлень: O(n log n)
- пам'ять на процес: O(1)

Переваги алгоритму:

- ефективний для кільцевої топології
- мала кількість повідомлень
- симетричність процесів
- відсутність централізованого керування

Алгоритм (опис)

1. Кожен процес має:
    - унікальний UID
    - статус active
    - лічильник раундів
2. Для раунду k = 0,1,2,... :
    - активний процес надсилає повідомлення зі своїм UID
    - повідомлення відправляється вліво і вправо
    - максимальна дистанція передачі = 2^k
3. При отриманні повідомлення процес:
    - якщо отриманий UID більший → переслати далі
    - якщо отриманий UID менший → відкинути повідомлення
    - якщо отримано власний UID → процес стає лідером
4. Якщо процес отримав більший UID:
    - він переходить у стан passive
    - більше не ініціює повідомлення
5. Після завершення фази:
    - процеси, які залишилися активними, переходять до наступного раунду
6. Алгоритм завершується, коли:
    - один процес отримує назад власний UID після обходу кільця
7. Виводяться результати:
    - UID лідера
    - кількість раундів
    - сумарна кількість повідомлень

Спосіб розв'язання

Для розв'язання задачі реалізовано алгоритм Hirschberg–Sinclair (HS), розроблений у 1980 році. Алгоритм призначений для синхронного кільця і виконується раундами, у яких всі активні процеси діють паралельно.

Ключова ідея - поступове збільшення зони перевірки. У раунді r кожен активний процес перевіряє, чи є він найбільшим серед сусідів на відстані 2^(r−1). Якщо так - процес виживає і переходить до наступного раунду з подвоєною дистанцією. Завдяки експоненційному зростанню TTL алгоритм завершується за O(log N) раундів і передає O(N log N) повідомлень - суттєво краще за наївний алгоритм LCR зі складністю $O(N^2)$.

Кожен процес у будь-який момент перебуває в одному з двох станів:

- **active** - бере участь у виборах, ініціює зонди;
- **passive** - вибув, але залишається у кільці як ретранслятор чужих повідомлень.

Деталі реалізації

Реалізацію виконано мовою Python. Симуляція є синхронною: кожен "такт" відповідає одному кроку пересилання повідомлення між сусідніми процесами.

Структури даних

Стан кільця описується:

```
self.uids   = [...]       # uid[i] — UID процесу i
self.active = [True, ...] # active[i] — чи активний процес i
```

Повідомлення представлено класом Message з полями:

Поле	Тип	Призначення
uid	int	UID відправника зонду
direction	str	напрямок руху (cw / ccw)
hops	int	кількість пройдених кроків
ttl	int	максимальна кількість кроків (2^(r−1))
is_reply	bool	True якщо це reply, False якщо probe

Черга повідомлень реалізована як список deque - по одному на кожен процес (inbox[i] містить повідомлення, що надійшли до процесу i у поточному такті).

Логіка раунду

На початку кожного раунду r кожен активний процес надсилає два probe - у напрямках CW та CCW - з hops=1 та ttl=2^(r−1):

```
for pid in active:
    for d in (CW, CCW):
        dest = self._next(pid, d)
        inbox[dest].append(Message(uid=self.uids[pid], direction=d, hops=1, ttl=ttl)
```

Для кожного активного процесу відстежується словник reply_received[pid] = [False, False], де перший елемент - чи отримано reply на CW-probe, другий - на CCW-probe.

Обробка probe

Коли probe з UID u надходить до процесу з UID my_uid:

```
u > my_uid  та  hops < ttl  → пересилаємо далі (hops + 1)
u > my_uid  та  hops == ttl → надсилаємо reply у зворотньому напрямку
u < my_uid                  → зонд відкидається, процес u вибуває
u == my_uid                 → probe повернувся до відправника, він виграв раунд
```

Неактивний процес при отриманні будь-якого повідомлення лише пересилає його далі, не обробляючи.

Обробка reply

Reply рухається у зворотньому напрямку відносно probe. Коли reply з UID u надходить до процесу:

```
u == my_uid → reply дійшов до відправника, знімаємо очікування для цього напрямку
u != my_uid → пересилаємо reply далі у тому ж напрямку
```

Важливий нюанс: CW-probe генерує reply, що летить назад у напрямку CCW. Тому коли reply з direction=CCW досягає відправника - це означає завершення CW-гілки:

```
if msg.direction == CCW:
    reply_received[pid][0] = True  # CW-probe завершено
else:
    reply_received[pid][1] = True  # CCW-probe завершено
```

Паралельність та виключення

Щоб коректно імітувати паралельну обробку, поточний і наступний inbox розділені: всі процеси читають з inbox і пишуть у next_inbox. Після обробки всіх повідомлень такту inbox = next_inbox.

Процеси-жертви збираються у список newly_dead і виключаються лише після того, як всі повідомлення поточного такту оброблені - щоб уникнути

ситуації, коли процес виключається раніше, ніж встиг переслати повідомлення.

Завершення

Раунд завершується, коли inbox повністю порожній. Якщо після цього залишився один активний процес - він оголошується лідером. Інакше починається новий раунд з подвоєним TTL.

Код: https://github.com/MoloZzz/distributed-systems-tests/tree/main/labs/lab1

Результати

Алгоритм протестовано для різної кількості процесів. UIDs генеруються випадково. Результати підтверджують теоретичну складність O(N log N) повідомлень та O(log N) раундів.

N	Раундів	Повідомлень	Теор. макс. повідомлень ($8 \cdot N \cdot \log_2 N$)
5	2	26	~46
10	2	59	~100
50	3–4	~350	~560
99	3	~500	~1050

Фактична кількість повідомлень виявляється значно меншою за теоретичний максимум, оскільки при випадкових UIDs більшість процесів вибувають вже у перших раундах.

Лідером завжди стає процес з максимальним UID, що підтверджує коректність реалізації.

Додаток (виконання коду)

```
$ python labs/lab1/hs.py
Введіть кількість процесів: 5

==================================================
      РЕЗУЛЬТАТИ HS-АЛГОРИТМУ
==================================================
  UID переможця     : 50
  Кількість раундів : 2
  Всього повідомлень: 26
==================================================

--- Деталі раундів ---
HS-алгоритм: N=5
UIDs: [11, 50, 3, 16, 43]

=============================================
Раунд 1  (TTL=1)
Активні: [(0, 11), (1, 50), (2, 3), (3, 16), (4, 43)]
  P0(uid=11) виключено
  P2(uid=3) виключено
  P3(uid=16) виключено
  P1(uid=50): reply [cw] отримано
  P1(uid=50): reply [ccw] отримано
  P3(uid=16): reply [cw] отримано
  P4(uid=43): reply [ccw] отримано
```

```
==================================================
      РЕЗУЛЬТАТИ HS-АЛГОРИТМУ
==================================================
  UID переможця     : 88
  Кількість раундів : 2
  Всього повідомлень: 55
==================================================

--- Деталі раундів ---
HS-алгоритм: N=10
UIDs: [82, 88, 59, 7, 45, 22, 32, 66, 47, 16]
```

```
==================================================
      РЕЗУЛЬТАТИ HS-АЛГОРИТМУ
==================================================
  UID переможця     : 493
  Кількість раундів : 2
  Всього повідомлень: 424
==================================================

--- Деталі раундів ---
HS-алгоритм: N=50
UIDs: [451, 175, 124, 56, 110, 123, 166, 405, 315, 20, 22, 472, 290, 81, 493, 342, 197, 409, 53, 34, 491, 42, 322, 375, 325, 272, 337, 52, 243, 195, 335, 350, 225, 27,
114, 89, 78, 252, 186, 401, 19, 486, 362, 341, 55, 248, 326, 217, 247, 51]
```

```
$ python labs/lab1/hs.py
Введіть кількість процесів: 99


==================================================
      РЕЗУЛЬТАТИ HS-АЛГОРИТМУ
==================================================
  UID переможця     : 988
  Кількість раундів : 3
  Всього повідомлень: 1006
==================================================
```